

1 Proof Techniques

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version 2026-10-12 03:14 H

Learning Outcomes

Unit 1: ***Proof Techniques***

1. Know logical *proof strategies* for proving implications, set inclusions, set equalities, and quantified statements.
2. Be able to use *mathematical induction* in simple proofs.
3. Know techniques for *proving termination* and *correctness* of procedures.

Outline

1 Proof Techniques

1.1 Digression: Random Shuffle

1.2 Proof Templates

1.3 Mathematical Induction

1.4 Correctness Proofs

1.1 Digression: Random Shuffle

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.

- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right?*

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

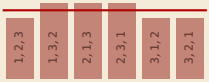
```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

← **WRONG! DO NOT USE**

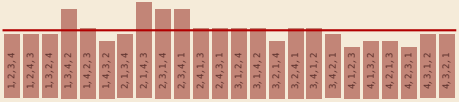
- Intuitively: *All elements are moved to a random index, so the order is random . . . right????*



$n = 2$



$n = 3$



$n = 4$

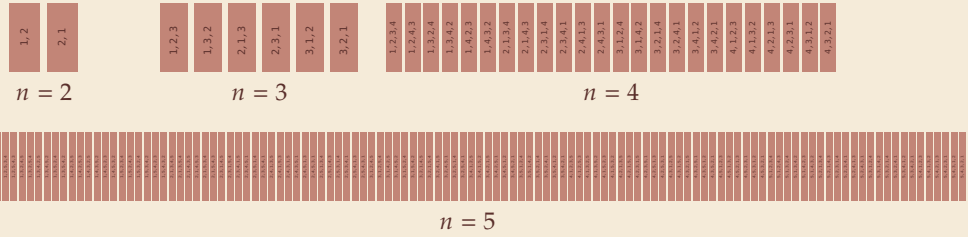


$n = 5$

Correct shuffle

- interestingly, a very small change corrects the issue

```
1 procedure shuffleKnuthFisherYates(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([i..n))
4         Swap A[i] and A[r]
5     end for
```



- looks good . . .
- . . . but how can we convince ourselves that it is correct, *beyond any doubt?*

1.2 Proof Templates

What is a *formal* proof?

A formal proof (in a logical system) is a **sequence of statements** such that each statement

1. is an *axiom* (of the logical system), or
2. follows from previous statements using the *inference rules* (of the logical system).

Among experts: Suffices to *convince a human* that a formal proof *exists*.

But: Use formal logic as guidance against faulty reasoning. $\rightsquigarrow$ bulletproof



Notation:

- Statements: $A \equiv$ "it rains", $B \equiv$ "the street is wet".
- Negation: $\neg A$ "Not A"
- And/Or: $A \wedge B$ "A and B"; $A \vee B$ "A or B or both"
- Implication: $A \Rightarrow B$ "If A, then B"; $\neg A \vee B$
- Equivalence: $A \Leftrightarrow B$ "A holds true *if and only if ('iff') B* holds true."; $(A \Rightarrow B) \wedge (B \Rightarrow A)$

Implications

To prove $A \Rightarrow B$, we can

- directly derive B from A *direct proof*
- prove $(\neg B) \Rightarrow (\neg A)$ *indirect proof, proof by contraposition*
- assume $A \wedge \neg B$ and derive a contradiction *proof by contradiction, reductio ad absurdum*
- distinguish cases, i. e., separately prove
  $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$. *proof by exhaustive case distinction*

Equivalences

To prove $A \Leftrightarrow B$,
we prove both implications $A \Rightarrow B$ and $B \Rightarrow A$ separately.

(Often, one direction is much easier than the other.)

Set Inclusion and Equality

To prove that a set S contains a set R, i. e., $R \subseteq S$,
we prove the implication $x \in R \Rightarrow x \in S$.

To prove that two sets S and R are equal, $S = R$,
we prove both inclusions, $S \subseteq R$ and $R \subseteq S$ separately.

1.3 Mathematical Induction

Quantified Statements

Notation

- Statements with parameters: $A(x)$ ≡ "x is an even number."
- Existential quantifiers: $\exists x : A(x)$ "There exists some x, so that $A(x)$."
- Universal quantifiers: $\forall x : A(x)$ "For all x it holds that $A(x)$."

  Note: $\forall x : A(x)$ is equivalent to $\neg \exists x : \neg A(x)$

Quantifiers can be nested, e. g., *ε-δ-criterion for limits*:

$$\lim_{x \to \xi} f(x) = a \qquad :\Leftrightarrow \qquad \forall \varepsilon > 0 \; \exists \delta > 0 \; : \; \big(|x - \xi| < \delta \big) \Rightarrow \big| f(x) - a \big| < \varepsilon .$$

To prove $\exists x : A(x)$, we simply list an example ξ such that $A(\xi)$ is true.

For-all statements

To prove $\forall x : A(x)$, we can

- derive $A(x)$ for an *"arbitrary but fixed value of x"*, or,
- for $x \in \mathbb{N}_0$, use ***induction***, i. e.,
  - prove $A(0)$, *induction basis*, and
  - prove $\forall n \in \mathbb{N}_0 : A(n) \Rightarrow A(n+1)$ *inductive step*

More general variants of induction:

- complete/strong induction
  inductive step shows $\big(A(0) \wedge \cdots \wedge A(n)\big) \Rightarrow A(n+1)$
- structural/transfinite induction
  works on any *well-ordered* set, e. g., binary trees, graphs, Boolean formulas, strings, ...

  no infinite strictly decreasing chains

1.4 Correctness Proofs

Formal verification

- verification: prove that a program computes the correct result

$\rightsquigarrow$ not our key focus in CS 566
but same techniques are useful for *reasoning* about algorithms

Here:

1. Prove that loop or recursive call eventually *terminates*.
2. Prove that a *loop* computes the *correct* result.

Proving termination

To prove that a recursive procedure $\texttt{proc}(x_1, \ldots, x_m)$ eventually terminates, we

- define a *potential* $\Phi(x_1, \ldots x_m) \in \mathbb{N}_0$ of the parameters
  (Note: $\Phi(x_1, \ldots x_m) \geq 0$ by definition!)
- prove that every recursive call decreases the potential, i. e.,
  any recursive call $\texttt{proc}(y_1, \ldots, y_m)$ inside $\texttt{proc}(x_1, \ldots, x_m)$ satisfies

$$\Phi(y_1, \ldots, y_m) < \Phi(x_1, \ldots, x_m) \qquad \text{which means}$$
$$\Phi(y_1, \ldots, y_m) \leq \Phi(x_1, \ldots, x_m) - 1$$

$\rightsquigarrow$ $\texttt{proc}(x_1, \ldots, x_m)$ terminates because
we can only strictly *decrease* the (integral) potential
a *finite* number of times from its initial value

- Can use same idea for a loop: show that potential decreases in each iteration.
  $\rightsquigarrow$ see tutorials for an example.

Loop invariants

Goal: Prove that a *post condition* holds after execution of a (terminating) loop.

```
1 // (A) before loop
2 while cond do
3     // (B) before body
4     body
5     // (C) after body
6 end while
7 // (D) after loop
```

For that, we

- find a *loop invariant* I (that's the tough part!)
- prove that I holds at (A)
- prove that $I \wedge cond$ at (B) imply I at (C)
- prove that $I \wedge \neg cond$ imply the desired post condition at (D)

Note: I holds before, during, and after the loop execution, hence the name.

Loop invariant – Example

- loop condition: $cond \equiv j < n$
- post condition (in line 13):
  $curMax = \max_{k \in [0..n-1]} A[k]$
- loop invariant:
  $I \equiv curMax = \max_{k \in [0..j-1]} A[k] \wedge j \leq n$

We have to proof:

(i) I holds at (A)

(ii) $I \wedge cond$ at (B) $\Rightarrow$ I at (C)

(iii) $I \wedge \neg cond \Rightarrow$ post condition

```
1  procedure arrayMax(A,n)
2      // input: array of n elements, n ≥ 1
3      // output: the maximum element in A[0..n − 1]
4      curMax := A[0]; j := 1
5      // (A)
6      while j < n do
7          // (B)
8          if A[j] > curMax
9              curMax := A[j]
10         j := j + 1
11         // (C)
12     end while
13     // (D)
14     return curMax
```

Loop invariant – Example